PLANNING DOCUMENT

Two rooms, one building.

A plan to split SkyForge into a squadron portal and a live campaign board, joined by a single cover page — and to give one administrator the ability to edit the squadron's own record from the browser, safely.

SCOPE	ADMIN	EST. EFFORT	ADDED COST
One squadron	Password, one holder	~29 h · 7 phases	$0.00 / mo

1 Summary

What is being proposed, and the single decision this document exists to support.

SkyForge today is one page answering one question: `what is happening on the server right now`. It does that well. It cannot answer the other question a squadron's website exists to answer — `who are we` — because that content has no author, no home, and no shape.

The proposal is to add a second room rather than a second site. A visitor arrives at a cover page that is already showing them the live front line, and chooses a door: **Dynamic Campaign** (the board as it exists, untouched) or **Squadron** (the unit's own record — charter, roster, qualifications, operations, comms, recruitment). Both rooms share one file, one deploy, one visual system.

The squadron's record is authored, not generated, so it needs an editor. You asked for a password. That is achievable and safe — but **only if the password is checked on a server, never in the page**. Section 4 explains why, and how to do it properly using infrastructure this project already runs.

> THE DECISION
>
> Approve the shape in sections 3 and 5, choose an option in section 4, and supply the eight inputs in section 11. Everything else is mine to execute.

1.1 What does not change

Nothing in the live board is rewritten. The Ops, Map, Planner, Pilots, Log and Files views keep their code, their data feeds and their behaviour. The publisher on the DCS box (`skyforge-upload.ps1`) is untouched. The existing test suites — 184 board checks and 65 planner checks — continue to pass, and are extended rather than replaced.

2 Settled constraints

Decisions already taken. These shape everything downstream and are not revisited below.

CONSTRAINT	DECISION	CONSEQUENCE FOR THE BUILD
Tenancy	`One squadron`	No unit switcher, no per-unit routing, no namespacing. Squadron identity is a single object at the root of one data file. Materially simpler.
Administration	`Single shared password`	No user accounts, no per-user permissions, no attribution by person. One secret, two privilege levels: read, or read-and-write. Anyone holding the password is you.
Hosting	`Unchanged`	Private S3 bucket behind CloudFront with Origin Access Control. No public bucket exists to misconfigure — which turns out to be the reason the password can be made safe.
Delivery	`Single file`	`index.html` stays one self-contained artifact. It grows from 204 KB to roughly 320 KB — still one request, still cached at the edge.

3 The shape

Why the split is real, why the cover page must not be a menu, and how the two rooms relate.

3.1 The seam is genuine

The two halves are not two categories of content. They differ in tempo and in authorship, which is the only justification a split like this ever needs:

	DYNAMIC CAMPAIGN	SQUADRON
Changes	`every 10 seconds`	`every few weeks`
Authored by	`the server`	`people`
Answers	What is happening now?	Who are we?
Truth source	`live.json · campaign.json · status.json`	`squadron.json`
Fails by	going stale	going unmaintained

3.2 The cover page is a cover, not a menu

The obvious build is a splash screen with two buttons. It is also the mistake. A page whose only content is asking where you want to go charges every visitor a click for zero information, and it is the most common way an idea like this ends up feeling cheap.

Instead the cover page is **already showing the war while you choose**: the live front line drawn faintly across the full width, the current zone tally, who is airborne, when the last sortie flew. The two doors sit on top of it as typography, not as buttons.

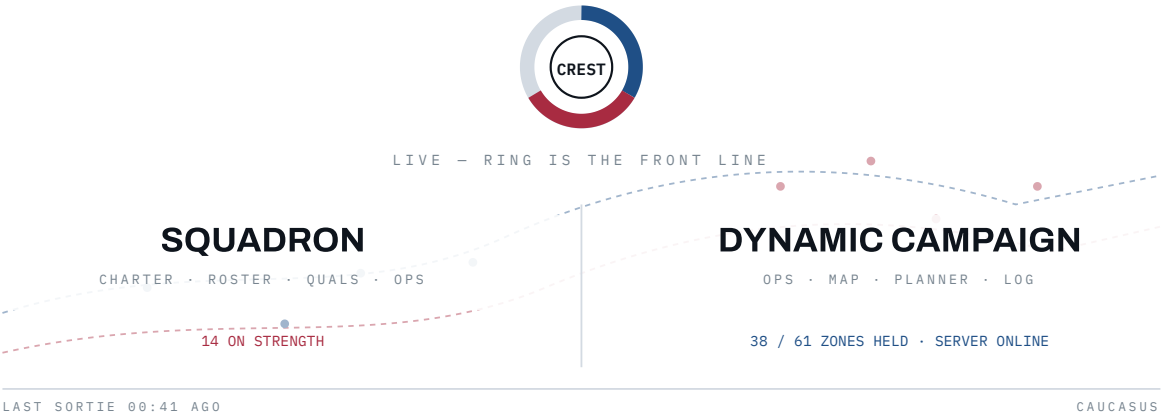


FIG 1 — COVER PAGE. EVERY VALUE SHOWN IS LIVE DATA THE BOARD ALREADY LOADS.

3.3 Choosing changes state, it does not navigate

There is no second page load. Choosing `Dynamic Campaign` brings the chart up to full contrast and assembles the instrument chrome around it. Choosing `Squadron` lets the chart recede to a watermark and brings the unit's identity forward. One continuous space in two lighting states, driven by the same `view` router the board already uses.

Consequences: the back button works, deep links work (`#squadron/roster`), the ten-second poll keeps running underneath, and there is no flash of empty page between rooms.

4 Administration — read this before approving anything

The password is the one part of this plan that can be built wrongly in a way that looks identical to being built correctly.

> THE PROBLEM, STATED PLAINLY
>
> The site is static files served to the public. Everything in `index.html` is readable by anyone who presses Ctrl-U. **A password checked inside the page is not a password** — it is a doorbell. Anyone can read the check, delete it, or call whatever it was guarding directly. This stays true no matter how the check is written, hashed, minified or obfuscated.
>
> So the question is not "how do we add a password". It is **"where does the write actually happen, and who holds the credential for it"**.

4.1 Three honest options

A No admin UI — edit in git, deploy FALLBACK

`squadron.json` lives in the repository. To change the roster you edit the file and run the deploy script. No password exists, so no password can leak.

Security	Perfect — there is no write path from the internet at all.
Cost	`$0.00`
Build	`0 h`
Against	Every edit needs a keyboard, the repo and a deploy. Does not meet the brief you gave.

B Password verified server-side in Lambda RECOMMENDED

The page never learns the password's stored form and never holds an AWS credential. It sends what you typed to an endpoint; a Lambda function compares it against a hash only it can read, and only that Lambda can write to the bucket.

Security	Sound. The only write path is the Lambda, and the only credential lives in the Lambda's execution role — never in the browser.
Cost	`$0.00 — inside the perpetual free tier at this volume`
Build	`~6 h`
Against	One more moving part. Mitigated by it being the `same` moving part you already run for the Discord bot.

C Cognito, or CloudFront signed cookies REJECTED

Real identity management, user pools, password-reset flows. Correct for a product with many users at many privilege levels. For one administrator with one password it is a large amount of configuration bought for nothing, and it adds a service you would then have to maintain.

4.2 How option B works, precisely

This is not new infrastructure. It is the pattern already deployed in `infra/bot.tf` for the Discord bot — an API Gateway HTTP API in front of a Lambda — pointed at a different job.

(Lambda Function URLs are blocked in this AWS account, which is why the bot uses API Gateway; the same applies here.)

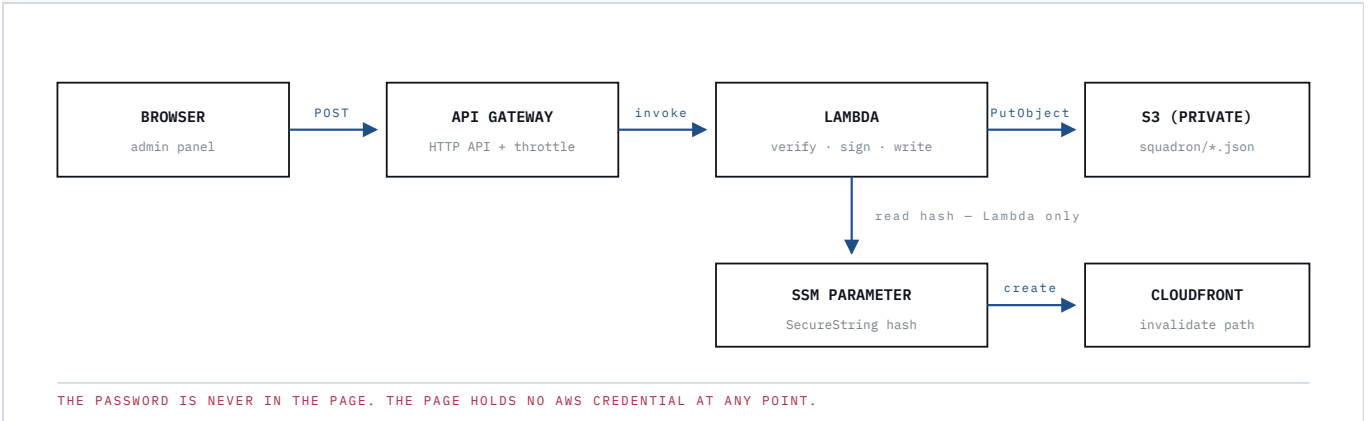


FIG 2 — ADMIN WRITE PATH. SAME LAMBDA + API GATEWAY PATTERN AS THE DISCORD BOT.

#	STEP	DETAIL
1	**You set the secret**	You run one AWS CLI command that stores a hash of your chosen password in SSM Parameter Store as a `SecureString`. **I never see the password.** You can change it any time by re-running that command — no deploy needed.
2	**Unlock**	The admin panel POSTs the typed password to the endpoint over HTTPS. The Lambda compares it against the stored hash using a constant-time comparison.
3	**Session token**	On success the Lambda returns a short-lived signed token (HMAC, two-hour expiry, signed with a second SSM secret). The password itself is not retained by the browser.
4	**Write**	Each save POSTs the edited section plus the token. The Lambda re-verifies the token, validates the payload against a schema, merges it into `squadron.json` and writes to S3.
5	**Publish**	The Lambda issues a CloudFront invalidation for that one path. The change is live in seconds.
6	**Versioning**	Every write also stores a timestamped copy under `squadron/history/`. A bad edit is recoverable, and you get a change log for free.

4.3 Hardening included in the six hours

- **Rate limiting** — API Gateway throttle plus a Lambda-side lockout: five failed unlock attempts locks further attempts for fifteen minutes.
- **Schema validation** — the Lambda rejects anything that is not the expected shape, so a malformed or hostile body cannot corrupt the file.
- **Size ceiling** — a hard cap on request body size, so the endpoint cannot be used to fill your bucket.
- **Scoped IAM** — the Lambda's role can write `only` the `squadron/` prefix. It cannot touch `data/`, `index.html`, or the campaign backups.
- **Output escaping** — every authored string is escaped on render. Authored content must never be able to inject script into the page.
- **No secret in git** — the password hash lives only in SSM, so the repository stays publishable.

WHAT THIS DOES NOT PROTECT AGAINST

Anyone holding the password can edit the squadron's record, and there is no per-person attribution. That follows directly from your decision in section 2 and is the right trade for a single-administrator site. If you later want to know `which` person made a change, that is a different design and I would say so before building it.

5 Information architecture

Every page in both rooms, what is on it, and where each field comes from. Nothing here is invented data.

5.1 Dynamic Campaign — unchanged

VIEW	SOURCE	CHANGE
Ops	`campaign.json · live.json`	None
Map	`zones.json · live.json`	None
Planner	`live + browser storage`	Gains one button: `publish as ops order` (section 5.2, Operations)
Pilots	`pilots.json · status.json`	Renamed **On Station**. Answers "who is airborne", not "who are we"
Log	`history.json · events.json`	None
Files	`static`	None

5.2 Squadron — new

Eight pages. The **Source** column is the important one: **authored** means a human writes it and it is editable in the admin panel; **live** means it is already being published by the server and is read-only.

#	PAGE	SOURCE	CONTENTS
1	**Charter**	`authored`	Crest, unit name and motto, formation date, theatre of operations, the CO's statement, rules of engagement, standing orders. The page that says who you are.
2	**Roster**	`authored + live`	Every pilot on strength. **Authored:** callsign, appointment (CO / XO / Flight Lead / Wingman), airframes, date joined, status, short bio. **Live:** IAF rank and shoulder board, XP, sorties, return rate — all already computed by the board today.
3	**Qualifications**	`authored`	A matrix of pilots against qualifications — type rating per airframe, air-to-air refuelling, night, CAS, SEAD, section lead, instructor — each cell carrying who signed it off and when. **This is the highest-value page in the plan** and is covered in 5.3.
4	**Operations**	`authored + planner`	The archive of ops orders. Each entry: name, date, the frag (imported straight from the Mission Planner's existing brief generator), participating flights, outcome, after-action notes. The squadron's history in its own words.
5	**Training**	`authored`	The syllabus. What a new pilot flies first, what each qualification requires, and how the progression maps onto the IAF rank ladder the board already awards.
6	**Comms**	`authored`	The kneeboard card: SRS frequencies, callsign allocation, IFF, check-in procedure. Small, unglamorous, and the page members will actually open mid-flight.
7	**Join**	`authored`	Discord invite, what modules you need, what is expected of you, the server address and how to connect.
8	**Admin**	`gated`	Hidden until unlocked. Turns pages 1–7 into editable fields in place, with a save per section and a revision list. Not a separate CMS — the same pages, in edit mode.

5.3 Why the qualifications matrix matters

Every DCS group's website has an about page and a member list. Almost none have a qualifications board, because it requires someone to decide what the unit actually trains for and

to sign names against it. That is exactly why it reads as a real squadron rather than a Discord server with a logo.

It is also the page that gives the ranks you already built somewhere to point. A rank earned purely by XP is a scoreboard; a rank alongside a signed-off qualification is a record of standing.

5.4 Where the pilots split

Pilots appear in both rooms, and the split is clean because the two rooms ask different questions about the same people:

QUESTION	ROOM	PAGE
Who are we?	`Squadron`	Roster — appointment, quals, bio, rank, career totals
Who is airborne?	`Dynamic Campaign`	On Station — slot, aircraft, altitude, session

A pilot's name is a link in both directions, so the roster entry and the live contact are one click apart.

6 Data model

One file holds everything a human authored. Everything else is already published by the server.

`squadron.json` is the single source of authored truth. It is deployed with the site, read on load like every other feed, and written only by the admin Lambda. Shape:

```
{
  "version": 7,
  "updated": "2026-08-25T18:40:00Z",

  // identity — the Charter page
  "unit": {
    "name": "No. 1 Squadron",
    "nickname": "Tigers",
    "motto": "...",
    "formed": "2026-06-14",
    "theatre": "Caucasus",
    "colour": "#C8452F",          // the one owned colour — section 7.2
    "discord": "https://discord.gg/...",
    "charter": "markdown...",
    "roe": "markdown..."
  },

  // people — the Roster page. Keyed by the in-game name the server reports,
  // which is what joins this to pilots.json with no matching logic.
  "pilots": {
    "GODZ 4-0-4": {
      "callsign": "GODZ",
      "appointment": "Commanding Officer",
      "joined": "2026-06-14",
      "status": "active",           // active | reserve | retired
      "airframes": ["F-16C", "A-10C II"],
      "bio": "..."
    }
  },

  // the Qualifications matrix
  "quals": {
    "codes": [
      { "id": "AAR",  "name": "Air-to-Air Refuelling", "group": "skill" },
      { "id": "SEAD", "name": "SEAD / DEAD",           "group": "role"  }
    ],
    "awards": [
      { "pilot": "GODZ 4-0-4", "qual": "AAR", "date": "2026-07-02", "by": "R00K" }
    ]
  },

  // the Operations archive — brief comes from the Planner, unchanged
  "operations": [
    { "id": "op-001", "name": "...", "date": "...",
      "brief": "...", "flights": [], "outcome": "...", "aar": "..." }
  ],

  "comms":    { "srs": [ { "label": "Squadron Common", "freq": "251.000", "mod": "AM" } ] },
  "training": "markdown...",
  "join":     { "requirements": "...", "modules": ["F-16C"] }
}
```

6.1 Two properties worth noting

- **The join key is the in-game name.** Pilots are keyed by exactly the string the server reports in `pilots.json`. No fuzzy matching, no separate ID to keep in sync — if a name appears in both files, the roster entry and the live stats are the same person.
- **Authored data never overrides live data.** XP, rank, sorties and return rate come only from the server. The admin panel cannot edit them, by design: the moment a roster can be hand-tuned, none of the numbers on the site mean anything.

7 Design direction

You asked for futuristic. This section argues for a specific reading of that word, and against the usual one.

7.1 The trap, named

When a flight-sim site is briefed as "futuristic", what gets built is almost always the same object: neon cyan on near-black, hexagon grids, glass panels, animated scanlines, Orbitron or Rajdhani, HUD brackets around every element. Every DCS group and Star Citizen org site looks like this. It reads as amateur to the people you most want to impress, and it dates in about eighteen months.

What actually reads as advanced is **density and precision** — a Jeppesen approach plate, an ATO, a real multi-function display. Nobody looks at a Honeywell cockpit display and thinks "dated". That is the register SkyForge already commits to, and it is the project's strongest asset.

7.2 One system, two registers

	DYNAMIC CAMPAIGN	SQUADRON
Reads as	an instrument	a document
Type	mono-dominant, small, dense — IBM Plex Mono carries the page	Archivo at display sizes, tight tracking, real hierarchy; mono reserved for data
Density	high — every pixel is telemetry	generous — this is prose, and prose needs air
Colour	`blue = own · rose = hostile · amber = attention · jade = ready`	the same, plus **one** owned squadron colour

Same two typefaces, same palette, two registers. That is how one product gets two rooms without looking like two websites glued together.

The squadron's own colour follows a precedent already in the stylesheet: the IAF shoulder-board colours are flagged there as `the only non-semantic colours on the board`, reproduced because they belong to a real artifact. A unit's colours are the same kind of thing. One colour, owned, documented, and never used to mean anything else.

7.3 The signature: a crest that is the front line

Every squadron site has a crest, and it is always a static image. Yours is drawn in SVG, and **its outer ring is the campaign**: one segment per zone, blue or rose by who currently holds it, redrawn on every poll. When the boys take Kutaisi, the crest changes.

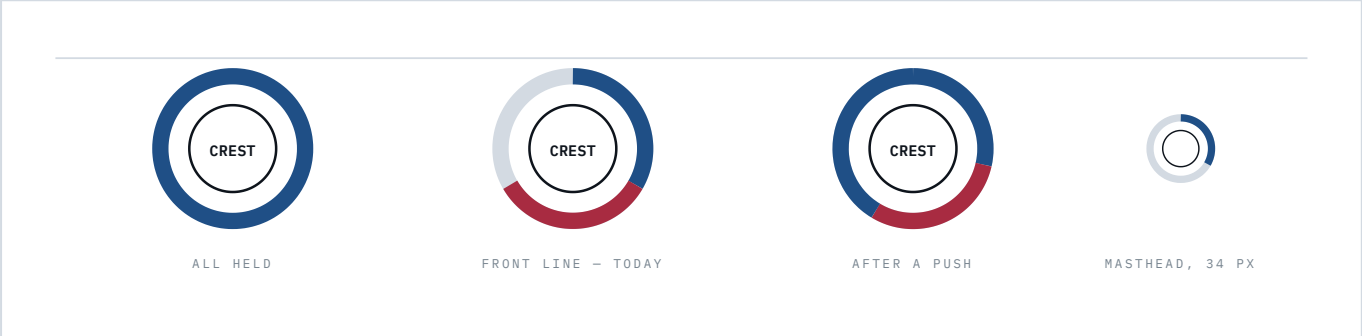


FIG 3 — THE LIVING CREST AT FOUR STATES. SAME OBJECT, FOUR SIZES, REAL DATA.

It scales the whole way down — full size on the cover, small in the masthead, tiny beside a pilot's shoulder board. The unit's identity and the state of the war become the same object. The data is already loaded, so it costs almost nothing to build, and I have not seen anyone do it.

7.4 Restraint

Spend the boldness in one place. The crest is that place. Everything around it stays disciplined: hairline rules, no gradients, no glass, no drop shadows, a 3 px radius because zero everywhere is its own tell.

Motion is one orchestrated moment — the cover-to-room transition — and nothing else. No ambient particles, no scanlines, no typing effects. The restraint is what makes it read as an instrument rather than a game menu, and it is why `prefers-reduced-motion` costs nothing to honour.

> **EXPLICITLY NOT BUILDING**
>
> Glassmorphism · neon accent on black · hexagon or circuit backgrounds · animated scanlines and glitch text · Orbitron, Rajdhani, Exo · HUD brackets as decoration · a hero with a big number and a gradient · stock fighter-jet photography · a loading bar that is not loading anything.

8 Build plan

Seven phases. Each ends in something you can look at, and each is verified before the next begins.

#	PHASE	EFFORT	DELIVERABLE	VERIFIED BY
0	**Decisions & content**	yours	The eight inputs in section 11	Nothing is built until these land
1	**Cover page & shell**	4 h	The cover with live data, both doors, both transitions, deep links, back button	Browser test at 1920 / 1366 / 414; back button returns to cover
2	**Static squadron pages**	4 h	Charter, Comms, Join, Training rendering from `squadron.json`	Every field renders; missing fields degrade rather than break
3	**Roster & quals**	5 h	The authored-plus-live merge, and the qualifications matrix	A pilot in one file but not the other renders correctly both ways
4	**Living crest**	3 h	The SVG crest at four sizes, driven by zone ownership	Ring segment count equals zone count; colours match the map exactly
5	**Admin**	6 h	Terraform for Lambda, API Gateway, SSM and IAM; the in-page edit mode	Wrong password rejected; lockout fires; a save round-trips and appears live
6	**Operations archive**	4 h	The ops-order archive, and `publish` in the Mission Planner	A plan built in the Planner appears as an ops order without retyping
7	**QA & deploy**	3 h	Test suites extended, responsive sweep, deploy and invalidate	All existing checks still pass, plus the new ones
	Total	**29 h**	My working estimate, not a quotation. Phases 1–4 are the visible half; 5 is the careful half.	

> **A SUGGESTED STOPPING POINT**
>
> Phases 1 to 4 give you a complete, good-looking squadron portal that only I can update — which is exactly option A in section 4. If phase 5 turns out not to be worth it once you see the site, you can stop at 4 and lose nothing. **I would build in that order for exactly this reason.**

8.1 One structural change worth making

At roughly 320 KB, `index.html` is still fine to ship as one file — it is one request and it caches at the edge. But it is uncomfortable to `edit` as one file. Phase 1 introduces a small build step that assembles the shipped file from separate sources, following the same pattern already used to splice in the Mission Planner. The deployed artifact does not change; only my working copy does.

9 Cost

The portal adds no meaningful running cost. This is the whole story.

SERVICE	USAGE AT YOUR VOLUME	RATE	MONTHLY
API Gateway HTTP API	< 1,000 requests	$1.00 / million	$0.00
Lambda	< 1,000 invocations	1 M free / month	$0.00
SSM Parameter Store	2 standard parameters	free	$0.00
CloudFront invalidations	< 50 paths	1,000 free / month	$0.00
S3 storage & requests	< 1 MB, negligible GETs	—	$0.00
Added to your bill			$0.00

The portal is served from the same CloudFront distribution as the board and does not depend on the EC2 instance at all. **It stays up when the server is off** — which is most of the month, and is arguably the point of having it.

10 Risks

Ordered by how much damage each does if ignored.

#	RISK	SEVERITY	MITIGATION
1	**A client-side password**	High	The failure mode is that it looks like it works. Section 4 is the mitigation: the check happens in Lambda or it does not happen. If phase 5 is ever rushed, cut the feature rather than the check.
2	**The record goes stale**	Medium	A roster nobody updates is worse than no roster — it advertises neglect. Mitigated by keeping the authored surface deliberately small, and by every page showing its own last-updated date, so staleness is visible rather than hidden.
3	**The crest does not land**	Medium	It is the one aesthetic bet in the plan. Contained to phase 4 and about three hours. If it does not work when you see it, a conventional crest drops into the same slot and nothing else changes.
4	**Authored HTML injection**	Medium	Authored content is rendered as text, never as markup. Escaping is applied at render, and covered by tests in phase 7.
5	**Cache serving a stale edit**	Low	An edit that does not appear looks like a broken save. The Lambda invalidates the exact path on every write, and `squadron.json` carries a short max-age like the other feeds.
6	**Two writers, one bucket**	Low	The DCS publisher writes `data/`; the admin Lambda writes `squadron/`. Different prefixes, and the Lambda's IAM role cannot reach the other one even by mistake.

11 What I need from you

Eight inputs. Nothing is built until these land, because guessing them would produce a portal about a squadron that does not exist.

#	INPUT	NOTES
1	**Unit name and nickname**	What the squadron is actually called.
2	**Motto and formation date**	The date can be the day the server first went up. That is a real founding.
3	**Crest**	If you have artwork, send it. If not, say so and I will draw three options in phase 4.
4	**Squadron colour**	One colour. If you have no view, I will propose three that sit correctly against the existing palette.
5	**Discord invite link**	A permanent invite, not a 24-hour one.
6	**Qualifications list**	What the squadron trains for. Start with six to eight; the matrix grows badly past about a dozen. I can propose a starting set from what your pilots actually fly.
7	**Appointments**	Who is CO, XO, flight lead. This is the one piece of the roster I genuinely cannot infer.
8	**Admin decision**	Option A or B from section 4. If B, you will set the password yourself with a command I will give you — **I will not know it.**

WHAT I DO NOT NEED

Finished prose. Charter, ROE, training and join text can all start as placeholders and be written through the admin panel once it exists — that is precisely what it is for. Only items 1 to 8 block the build.

12 Verification

How you will know it works, without taking my word for it.

PHASE	CHECK	PASS CONDITION
1	Open the site, choose each door, press back	Cover shows live values; both rooms open without reload; back returns to the cover
1	Open `#squadron/roster` directly	Lands on the roster, not the cover
2	Delete a field from `squadron.json`	The page renders without it; nothing throws
3	A pilot in `pilots.json` but not the roster	Appears with live stats and no authored fields — no blank row, no crash
4	Compare crest ring against the ops map	Segment count equals zone count; every colour matches
5	Enter a wrong password six times	Rejected, then locked out for fifteen minutes
5	View source, search for the password	**Not present anywhere in the page or any response**
5	Edit the charter, save, reload	Change is live within seconds; previous version listed in history
6	Build a plan in the Planner, publish it	Appears in the ops archive with no retyping
7	Run the full test suite	184 board + 65 planner checks still pass, plus the new ones
7	Open on a phone	No horizontal scroll; the matrix scrolls inside its own container

NEXT STEP

Tell me which parts of this you want changed, answer section 11, and pick A or B in section 4. If you would rather see it than read about it, I can build phase 1 alone — the cover page, with real data and both transitions working — and you can decide the rest looking at the real thing.